

Course CS367

Reproduction of “INSIDE: LLMs’ Internal States Retain the Power of Hallucination Detection”

Chen et al., ICLR 2024

Project Report

Name	Student ID
Aayaan Ali Khan	202411001
Shlok Bhavsar	202411018
Prapti Chotaliya	202411030

October 3, 2026

Abstract

We reproduce the method of *INSIDE: LLMs’ Internal States Retain the Power of Hallucination Detection* (Chen et al., ICLR 2024). The paper proposes two components: *EigenScore*, a hallucination score computed from the covariance of sentence embeddings taken from the middle layer of a language model, and *test-time feature clipping*, which truncates extreme activations of the penultimate layer. The authors’ public repository could not be executed as released, and it does not contain the feature-clipping component at all. We repaired twelve blocking defects, implemented feature clipping from the paper’s description, added support for running a 7B model on an 8 GB GPU, and wrote an automated test suite. We then evaluated on 250 CoQA questions using Llama-2-7B in 4-bit precision. The ordering of the five detection methods reported in the paper is reproduced exactly, with EigenScore best (AUROC 83.5 without and 82.5 with clipping). The effect of feature clipping could not be resolved at this sample size: a paired bootstrap gives a change of -1.0 points with a 95% interval of $[-4.7, +2.6]$.

Contents

1	Introduction	2
2	The Method	2
2.1	EigenScore	2
2.2	Test-time feature clipping	2
2.3	Baselines and evaluation	3
3	Implementation	3
3.1	State of the original code	3
3.2	Feature clipping	3
3.3	Running a 7B model on an 8 GB GPU	4
3.4	Tests	4
4	Experimental Setup	4
5	Results	5
5.1	Comparison of detection methods	5
5.2	EigenScore against the strongest baseline	6
5.3	Feature clipping	6
6	Discussion and Limitations	6
7	Conclusion	7
A	Contents of the Submission	7
B	Reproduction Instructions	7

1 Introduction

Large language models (LLMs) frequently produce fluent but factually incorrect statements, commonly called *hallucinations*. Detecting such outputs without external fact-checking is important for deployment. Existing approaches operate either on token probabilities (perplexity, entropy) or on the decoded text (agreement between several sampled answers). Both discard information that is still present inside the network.

The selected paper, INSIDE [1], instead measures the disagreement among several sampled answers *in the model's internal embedding space*. This report documents our implementation and reproduction of that work for the course CS367. Our contributions are:

1. A working, documented version of the authors' code base (twelve blocking defects repaired).
2. An implementation of test-time feature clipping (Section 3.2 of the paper), which is absent from the public repository.
3. Memory engineering that allows the pipeline to run on a consumer GPU with 8 GB of memory.
4. An automated test suite of eight tests.
5. An experimental reproduction on CoQA, with statistical analysis of the results.

2 The Method

2.1 EigenScore

For a question x , the model is sampled K times, producing responses $y_1, \dots, y_K$. For each response, the sentence embedding $z_i \in \mathbb{R}^d$ is the hidden state of the last generated token at the middle layer. Let $Z = [z_1, \dots, z_K] \in \mathbb{R}^{d \times K}$ and let $J_d = I_d - \frac{1}{d}\mathbf{1}\mathbf{1}^\top$ be the centering matrix. The covariance matrix is

$$\Sigma = Z^\top J_d Z \in \mathbb{R}^{K \times K}, \quad (1)$$

and the EigenScore is

$$\mathcal{E}(Y \mid x, \theta) = \frac{1}{K} \log \det(\Sigma + \alpha I_K) = \frac{1}{K} \sum_{i=1}^K \log \lambda_i, \quad (2)$$

where λ_i are the eigenvalues of $\Sigma + \alpha I_K$ and $\alpha = 10^{-3}$ is a regulariser that keeps the matrix full rank. Up to additive constants this quantity is the differential entropy of a Gaussian fitted to the embeddings. If the model is confident, the K embeddings nearly coincide, most eigenvalues collapse to the floor α , and the score is low. If the model is guessing, the embeddings spread out and the score is high. A larger score therefore indicates a more probable hallucination.

2.2 Test-time feature clipping

Self-consistent hallucinations, where the model repeats the same wrong answer, are invisible to any consistency measure. The paper hypothesises that extreme activations in the penultimate layer contribute to such overconfidence and truncates them elementwise:

$$\text{FC}(h) = \begin{cases} h_{\min}, & h < h_{\min} \\ h, & h_{\min} \leq h \leq h_{\max} \\ h_{\max}, & h > h_{\max} \end{cases} \quad (3)$$

The thresholds $h_{\min}$ and $h_{\max}$ are the bottom and top p -th percentiles of each neuron, estimated from a memory bank of the most recent N token activations. The paper uses $p = 0.2\%$ and $N = 3000$.

2.3 Baselines and evaluation

We compare against the paper’s four baselines: *Perplexity*, *Length-Normalised Entropy* (LN-Entropy), *Lexical Similarity* (mean pairwise ROUGE-L between the K answers) and *Energy*. A response is labelled correct when its ROUGE-L against the reference answer exceeds 0.5 (the paper’s AUC_r). Methods are compared by the area under the ROC curve (AUROC) of the score as a predictor of incorrectness.

3 Implementation

3.1 State of the original code

The released repository (alibaba/eigenscore, commit ea8062a) has empty setup and run sections and could not be executed. Table 1 lists the blocking defects we repaired. Every original file is preserved next to the repaired one with the suffix `.orig`.

Table 1: Blocking defects in the released code and their repairs.

#	File	Problem	Repair
1	pipeline/generate.py	Imports non-existent module <code>dataeval.TruthfulQA</code>	Import removed
2	func/metric.py	<code>np.float</code> (removed in NumPy 1.24) used 14 times	Replaced by <code>np.float64</code>
3	_settings.py	Absolute paths of the authors’ machine	Repository-relative, overridable by environment variables
4	pipeline/generate.py	Log opened before its directory exists	Directory created first
5	func/evalFunc.py	Imports only valid from inside <code>func/</code>	Package-qualified imports
6	func/metric.py	<code>selfcheckgpt</code> imported at load time, used by dead code	Import moved into the function
7	pipeline/generate.py	BERTScore model built from a missing local path for a disabled metric	Construction removed
8	dataeval/*	<code>import ipdb</code> at module level	Commented out
9	func/plot.py	Saves to a missing directory, then blocks on <code>plt.show()</code>	Agg backend, <code>makedirs</code> , no <code>show()</code>
10	func/evalFunc.py	<code>split("_")[1]</code> fails for ordinary paths	Whole path passed on
11	func/metric.py	<code>.to("cuda")</code> hard-coded 16 times	Device derived from the tensors
12	dataeval/load.py	<code>DEFAULT_DEVICE='cuda:7'</code>	Left unchanged (off the execution path)

An additional defect was found at run time. When generations were split into several forward passes, the original code recomputed EigenScore inside the batch loop, so only the last batch (2 of the 10 answers) contributed. We pool the embeddings of all batches before forming the covariance matrix; a test confirms equivalence with the original function in the single-batch case.

3.2 Feature clipping

The module `func/feature_clip.py` implements Section 3.2 of the paper. The class `FeatureClipper` maintains a ring buffer of N token activations, computes per-neuron percentile thresholds, and applies Equation (3). It is attached as a forward hook on the penultimate decoder layer, so that the clipping acts during autoregressive decoding and influences the generated text. Until

the bank holds a minimum number of rows the hook is a pass-through, so thresholds are never estimated from a few tokens.

3.3 Running a 7B model on an 8 GB GPU

- **4-bit weights** (NF4, bitsandbytes): a 7B model needs about 3.9 GB instead of 13.5 GB.
- `--num_return_sequences`: the $K = 10$ samples are produced in several smaller forward passes, keeping the paper’s value of K .
- `--low_memory`: requesting all hidden states retains every layer for every token, which ran out of memory on long CoQA stories. Because EigenScore reads only one layer, a forward hook captures only the middle layer. A test verifies the result is identical to the full path.

3.4 Tests

The file `tests/test_setup.py` contains eight tests, all of which pass: imports; clip thresholds agree with `numpy.percentile`; clipping implements Equation (3); ring-buffer wrap-around and warm-up; pooled EigenScore equals the original function; EigenScore separates consistent from diverse responses; the low-memory path equals the full-hidden-state path; decoder-layer discovery.

4 Experimental Setup

Table 2: Our setup compared with the paper’s.

	Paper	This work
Model	LLaMA-7B (fp16)	Llama-2-7B (4-bit NF4)
Dataset	CoQA, 7983 pairs	CoQA, 250 questions
Hardware	—	NVIDIA RTX 4060 Laptop, 8 GB
K / temperature / top- p	10 / 0.5 / 0.99	10 / 0.5 / 0.99
top- k	5	10 (repository default)
α / p / N	10^{-3} / 0.2% / 3000	10^{-3} / 0.2% / 3000
Embedding	middle layer, last token	middle layer, last token
Correctness	ROUGE-L (AUC_r) and sentence similarity (AUC_s)	ROUGE-L (AUC_r) only

The deviations are forced by the 8 GB GPU and by the unavailability of the `nli-roberta-large` model, which is needed for AUC_s . The claim tested is therefore the *ordering* of the methods, not the exact numbers. The model answered 36.8% of the 250 questions correctly. Two runs were made with an identical seed (2023) and identical questions: one without and one with feature clipping.

```
python -m pipeline.generate --model llama2-7b-hf --dataset coqa \
    --load_in_4bit --low_memory --feature_clip --no_sentsim \
    --num_generations_per_prompt 10 --num_return_sequences 2 \
    --fraction_of_data_to_use 0.03132 --seed 2023 --project_ind 10
python -m func.evalFunc data/output/llama2-7b-hf_coqa_10/0.pkl --correctness rouge
```

5 Results

5.1 Comparison of detection methods

Table 3: AUROC (%) on 250 CoQA questions, AUC_r . The paper’s column is LLaMA-7B, Table 1.

Method	Paper	This work	This work +FC
Energy	54.7	54.7	53.1
Perplexity	68.3	72.3	71.4
LN-Entropy	73.6	77.5	77.3
Lexical Similarity	77.8	80.5	79.3
EigenScore	80.8	83.5	82.5

The ranking $\text{EigenScore} > \text{Lexical Similarity} > \text{LN-Entropy} > \text{Perplexity} > \text{Energy}$ is identical to the paper’s. Energy, an out-of-distribution detector not designed for this task, is close to chance.

The paper’s EigenScore figure of 80.8 already includes feature clipping (its Table 2 reports 79.3 without and 80.4 with clipping in AUC_s), whereas its baselines are without clipping. The like-for-like comparison for 80.8 is therefore our clipped run, 82.5.

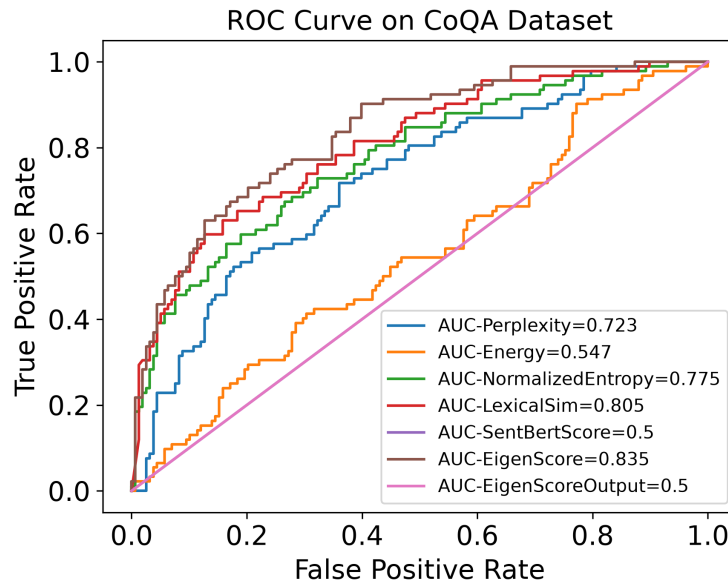


Figure 1: ROC curves on CoQA, without feature clipping. The entries “SentBertScore” and “EigenScoreOutput” are placeholders at 0.5 because the sentence-similarity model was not used.

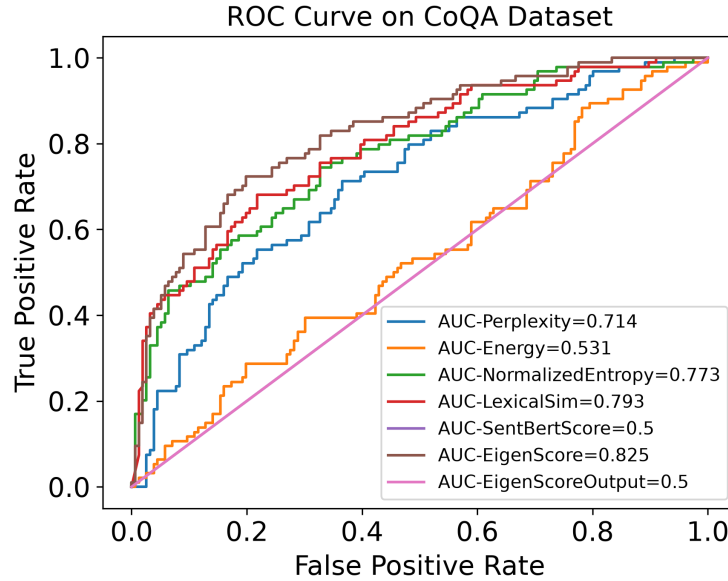


Figure 2: ROC curves on CoQA, with feature clipping.

5.2 EigenScore against the strongest baseline

A paired bootstrap over questions gives

$$\Delta\text{AUROC}(\text{EigenScore} - \text{LexSim}) = +0.0293, \quad 95\% \text{ CI} = [-0.0057, +0.0651], \quad P(> 0) = 0.95.$$

This agrees in sign and magnitude with the paper’s advantage of about +3 points on CoQA, and falls just short of excluding zero at 250 questions.

5.3 Feature clipping

The paper reports that clipping improves EigenScore on CoQA by +1.1 points (AUC_s). We measured a change of

$$\Delta\text{AUROC} = -0.0102, \quad 95\% \text{ CI} = [-0.0468, +0.0263], \quad P(\text{FC helps}) = 0.30$$

(10 000 paired bootstrap resamples, identical questions and seed). The interval contains both zero and the paper’s +0.011, so our experiment neither confirms nor contradicts the claim; it lacks the resolution to do so. The paper used 7983 questions. Clipping was verifiably active: 13 749 839 activations were clipped out of $1\,159\,186 \times 4096$, i.e. 0.29% (at most about 0.4% is expected from $p = 0.2\%$ per tail), and the thresholds are unit-tested against `numpy.percentile`. We also measured AUC_r where the paper reports AUC_s , so the two quantities are not strictly the same.

6 Discussion and Limitations

- **Sample size.** 250 questions give confidence intervals of roughly ± 4 points; effects of about one point cannot be resolved. A full run (`--fraction_of_data_to_use 1`) takes about 13 hours per configuration on our GPU.
- **Model and precision.** Llama-2-7B in 4 bits differs from the paper’s LLaMA-7B in fp16, so absolute values are not directly comparable.
- **Correctness measure.** Only ROUGE-L correctness was evaluated. The EigenScore-Output baseline and AUC_s require `nli-roberta-large`, which was unavailable.

- **Sampling.** Top- k was 10 rather than the paper’s 5.
- **Logarithm base.** The authors’ code uses $\log_{10}$ where Equation (2) writes $\log$. This is a constant factor and cannot change AUROC or the sign of correlations; we left it unchanged to match published numbers.
- **Inherent limits of the method.** It requires access to internal states (so not black-box APIs), costs K generations per query, and detects but does not correct hallucinations.

7 Conclusion

We obtained a working implementation of the INSIDE method, including the feature-clipping component that the authors’ repository lacks, and reproduced the paper’s central empirical finding: the relative ordering of the five hallucination detectors, with EigenScore the strongest. The additional claim that feature clipping improves EigenScore could not be tested conclusively with our compute budget; the implementation is verified by tests and shown to be active, and a full-scale run is the natural next step.

A Contents of the Submission

```
CS367_Submission/  
report/report.pdf, report.tex this document  
INSIDE_paper_...pdf the original paper  
source_code/  
  pipeline/generate.py sampling + score computation  
  func/metric.py EigenScore and baselines  
  func/feature_clip.py feature clipping (new)  
  func/evalFunc.py, plot.py evaluation, ROC plots  
  models/, dataeval/, utils/ model loading, datasets  
  tests/test_setup.py 8 automated tests  
  data/output/ raw results (.pkl), logs, figures  
  README.md, CHANGES.md, RESULTS.md, requirements.txt, run.sh
```

B Reproduction Instructions

```
cd source_code  
uv venv --python 3.11 .venv && source .venv/bin/activate  
uv pip install torch --index-url https://download.pytorch.org/whl/cu124  
uv pip install -r requirements.txt  
python -m tests.test_setup # 8/8 should pass  
bash run.sh # generation + evaluation
```

A CUDA GPU is required. CoQA and SQuAD are public datasets and must be placed under `data/datasets/` (`coqa-dev-v1.0.json`, `dev-v2.0.json`); they are omitted from the submission to keep it small.

References

- [1] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, J. Ye. INSIDE: LLMs’ Internal States Retain the Power of Hallucination Detection. *ICLR*, 2024. <https://arxiv.org/abs/2402.03744>

[2] Official code: <https://github.com/alibaba/eigenscore>